SEO Gap Analyzer

System Standard Operating Procedure

Full Technical Guide: HTML UI + n8n Backend Integration

Real-Time Progress Polling | Node-by-Node Workflow Setup | Troubleshooting

Version 2.0 | April 2026 | Confidential

1. Project Overview

The SEO Gap Analyzer is a web-based tool that allows users to compare their website against a competitor and receive an AI-generated keyword gap report directly in their email inbox. The frontend is a single-page HTML application; all backend processing is handled by an n8n automation workflow in the cloud.

1.1 The Core Problem (Old System)

Problem Statement

The previous architecture had a critical mismatch between the UI and the backend.

When a user submitted the form, the n8n webhook immediately returned HTTP 200 OK. The HTML interpreted this instant response as "job complete" and ran a fake 5-second animation — then showed the Thank You page.

In reality, the actual work (website crawling, AI analysis, email delivery) was still running in n8n and would take approximately 60–90 seconds to complete.

Result: Users saw a success screen before anything had actually happened.

1.2 The Solution (New System)

The new system uses a Polling Architecture. Instead of guessing when n8n is done, the HTML frontend continuously asks n8n every 4 seconds: "What is the current status?" The UI only advances to the next step and eventually shows the Thank You page when n8n itself confirms the job is complete.

Old System	New System (v2)
Fake timed animation (5–6 seconds)	Real-time polling every 4 seconds
UI independent of actual backend progress	UI synced exactly to n8n node execution
Thank You shown before email is sent	Thank You shown only after n8n confirms "done"
No jobId — no way to track the job	n8n returns jobId; frontend tracks it
Animation ends even if n8n fails silently	Errors in n8n surface immediately on the UI

2. System Architecture

The system consists of three components working together: the HTML frontend, n8n Workflow 1 (the main trigger), and n8n Workflow 2 (the status checker).

2.1 Component Overview

Component	Technology	Responsibility
HTML Frontend	Single HTML file (Vanilla JS)	Collects user input, shows real-time progress, polls n8n for status updates
n8n Workflow 1	n8n Webhook + Nodes	Receives form data, generates jobId, runs crawling + AI + email asynchronously
n8n Workflow 2	n8n Webhook + Code Node	Receives poll requests with jobId, returns current step and status as JSON

2.2 Data Flow Diagram

The following describes the complete sequence of events when a user submits the form:

```
USER BROWSER                              n8n CLOUD
_____________________________             ________________________________________


Step 1:  [Form Submit]  ——— POST ———▶ [Workflow 1: Webhook Trigger]
                                        • Generate unique jobId
                                        • Save status: step=1, running
         [Receive jobId] ◀—— JSON ——    • Return { jobId } immediately
                                        • Continue async work in background

Step 2:  [Poll: GET /status?jobId=X]  ▶ [Workflow 2: Status Check Webhook]
         [Receive: step=2, running]  ◀   • Look up jobId in Google Sheets
         → UI shows Step 2 active         • Return current status JSON

Step 3:  [Poll again after 4s]        ▶ [Status Check again...]
         [Receive: step=3, running]  ◀   • n8n has advanced to Step 3
         → UI shows Step 3 active

  ...continues every 4 seconds...

Final:   [Poll again]                 ▶ [Status Check]
         [Receive: status="done"]     ◀   • n8n finished, email sent
         → UI shows Thank You page
```

3. HTML Frontend — Setup & Configuration

The frontend is a single self-contained HTML file (seo-gap-analyzer-v2.html). No framework, no build step, no dependencies. It can be hosted on any web server or static file host.

3.1 The Only Two Lines You Must Edit

Quick Setup

Open the HTML file and find these two constants near line 242.
Replace the placeholder URLs with your actual n8n webhook URLs.

```
// 1. Main webhook — the form data is POST-ed here
const WEBHOOK_URL = 'https://YOUR-N8N.app.n8n.cloud/webhook/seo-gap-analysis';

// 2. Status check webhook — the UI polls this every 4 seconds
const STATUS_POLL_URL = 'https://YOUR-N8N.app.n8n.cloud/webhook/seo-gap-status';
```

3.2 How the Frontend Logic Works

The frontend JavaScript follows this decision tree after the form is submitted:

1. User submits form → Form validation runs → If valid, call go() function
2. POST request sent to WEBHOOK_URL with: myWebsite, competitorWebsite, email
3. n8n responds instantly with a jobId (e.g. { "jobId": "job_1234_abc" })
4. If jobId is received → Start real polling mode (pollForCompletion function)
5. If NO jobId received → Start fallback animation mode (60-90 second simulation)
6. Polling: every 4 seconds, GET request to STATUS_POLL_URL?jobId=XYZ
7. Each poll response updates the step indicators and progress bar on screen
8. When status = "done" → Stop polling → Show Thank You page
9. When status = "error" → Stop polling → Show Error page with message

3.3 What the UI Steps Mean

UI Step	What is Happening in n8n
Step 1 — Request sent to AI engine	Webhook received the form data; jobId was generated and returned
Step 2 — Crawling your website	n8n HTTP Request node is fetching your website URLs and page data
Step 3 — Scanning competitor keywords	n8n is fetching and parsing competitor website for keywords
Step 4 — Identifying content gaps	Google Gemini / AI node is running gap analysis on both datasets
Step 5 — Sending report to email	Gmail node is composing and sending the final HTML email report

3.4 Polling Configuration Options

Constant	Default Value
POLL_INTERVAL_MS	4000 (poll every 4 seconds)
MAX_POLL_TIME_MS	180000 (stop after 3 minutes)

To change polling frequency, edit these constants in the HTML file. Faster polling = more responsive UI but more n8n load. Slower polling = less load but less smooth animation.

4. n8n Workflow 1 — Main Trigger

This is your existing workflow. You need to make two modifications to it: (1) add a Code Node right after the Webhook node to generate a jobId and return it immediately, and (2) add status-update Code Nodes after each major processing step.

4.1 Full Node List (in order)

Node #	Node Type & Name
1	Webhook — "Receive SEO Request"
2	Code — "Generate JobId & Respond" ← NEW
3	Respond to Webhook — "Send JobId" ← NEW
4	Code — "Update Status: Step 2" ← NEW
5	HTTP Request — "Crawl Target Website" (existing)
6	Code — "Update Status: Step 3" ← NEW
7	HTTP Request — "Crawl Competitor Site" (existing)
8	Code — "Update Status: Step 4" ← NEW
9	Google Gemini Chat Model — AI Analysis (existing)
10	Code — "Update Status: Step 5" ← NEW
11	Gmail — "Send Report Email" (existing)
12	Code — "Update Status: Done" ← NEW

4.2 Node 1: Webhook — "Receive SEO Request"

This is your existing webhook trigger node. You only need to change one setting:

- Node Type: Webhook
- HTTP Method: POST
- Path: seo-gap-analysis
- **Response Mode: Change to "Respond to Webhook"** ← This is the critical change
- Authentication: None (or your existing setting)

Why "Respond to Webhook" mode?

In the default mode, n8n waits for the entire workflow to finish before sending a response. This causes the browser to hang for 60-90 seconds waiting for a reply.

"Respond to Webhook" mode lets you insert a "Respond to Webhook" node anywhere in the flow, so you can respond instantly with the jobId, then continue processing.

4.3 Node 2: Code — "Generate JobId & Respond"

Add this new Code node immediately after the Webhook node. It generates a unique job identifier, saves the initial status to Google Sheets, and prepares the response data.

Node Settings:

- Node Type: Code
- Mode: Run Once for All Items
- Language: JavaScript

Code to paste:

```
// Generate a unique job ID
const jobId = 'job_' + Date.now() + '_' + Math.random().toString(36).substr(2, 9);

// Store the jobId and initial status in the workflow data
// This will be used later by status-update nodes
const inputData = $input.first().json;

return [{
  json: {
    jobId: jobId,
    myWebsite: inputData.myWebsite,
    competitorWebsite: inputData.competitorWebsite,
    email: inputData.email,
    status: 'running',
    step: 1,
    message: 'Request received and queued'
  }
}];
```

4.4 Node 3: Respond to Webhook — "Send JobId"

This node sends the instant response back to the browser. After this node, the workflow continues running in the background — the browser already has the jobId it needs to start polling.

Node Settings:

- Node Type: Respond to Webhook
- Respond With: JSON
- Response Body: paste the JSON below

```
{
  "jobId": "{{ $json.jobId }}",
  "status": "accepted",
  "message": "Analysis started"
}
```

Important

After this "Respond to Webhook" node fires, the browser receives the jobId and immediately starts polling. The remaining nodes (4 through 12) continue executing asynchronously. The user sees live progress updates as each node runs.

4.5 Status Update Nodes (Nodes 4, 6, 8, 10, 12)

Add one of these Code nodes before each major processing step. They write the current status to Google Sheets so that Workflow 2 can read and return it to the polling UI.

Node 4 — Code "Update Status: Step 2" (before website crawl):

```
const data = $input.first().json;
data.step = 2;
data.status = 'running';
data.message = 'Crawling your website...';
return [{ json: data }];
```

Node 6 — Code "Update Status: Step 3" (before competitor crawl):

```
const data = $input.first().json;
data.step = 3;
data.status = 'running';
data.message = 'Scanning competitor keywords...';
return [{ json: data }];
```

Node 8 — Code "Update Status: Step 4" (before AI analysis):

```
const data = $input.first().json;
data.step = 4;
data.status = 'running';
data.message = 'AI is identifying content gaps...';
return [{ json: data }];
```

Node 10 — Code "Update Status: Step 5" (before email):

```
const data = $input.first().json;
data.step = 5;
data.status = 'running';
data.message = 'Preparing your email report...';
return [{ json: data }];
```

Node 12 — Code "Update Status: Done" (after email is sent):

```
const data = $input.first().json;
data.step = 5;
data.status = 'done';
data.message = 'Your report has been sent!';
return [{ json: data }];
```

4.6 How to Save Status to Google Sheets

Between each Code node and the next processing node, add a Google Sheets node to write the current status. This is the "database" that Workflow 2 reads from.

Google Sheets Node Settings (for writing):

- Node Type: Google Sheets
- Operation: Append or Update Row
- Sheet: Create a sheet called "seo_job_status"
- Columns to write: jobId, status, step, message, updatedAt
- Match column for update: jobId

Google Sheet structure (create this manually once):

Column Name	Type	Example Value
jobId	Text	job_1712345678_abc123
status	Text	running
step	Number	3
message	Text	Scanning competitor keywords...
updatedAt	Text	2026-04-06T12:00:00Z

5. n8n Workflow 2 — Status Check

This is a new, separate, small workflow. Its only job is to receive a poll request from the browser, look up the jobId in Google Sheets, and return the current status. It runs dozens of times per job — it must be fast and simple.

5.1 Full Node List

Node #	Node Type & Name
1	Webhook — "Status Poll Receiver"
2	Google Sheets — "Look Up Job Status"
3	Code — "Format Response"
4	Respond to Webhook — "Return Status JSON"

5.2 Node 1: Webhook — "Status Poll Receiver"

- Node Type: Webhook
- HTTP Method: GET
- Path: seo-gap-status
- Response Mode: Respond to Webhook
- Authentication: None

The browser will call this as: GET /webhook/seo-gap-status?jobId=job_1234_abc

The jobId is available in the webhook as: {{ $json.query.jobId }}

5.3 Node 2: Google Sheets — "Look Up Job Status"

- Node Type: Google Sheets
- Operation: Get Row(s) by Filter
- Sheet: seo_job_status (same sheet Workflow 1 writes to)
- Filter Column: jobId
- Filter Value: {{ $json.query.jobId }}
- Max Rows: 1

5.4 Node 3: Code — "Format Response"

This node formats the Google Sheets row data into the exact JSON format the HTML frontend expects.

```
const row = $input.first().json;

// If jobId not found in Sheets, return a safe default
if (!row || !row.jobId) {
  return [{
```

```
    json: { status: 'running', step: 1, message: 'Processing...' }
  }];
}

return [{
  json: {
    jobId:   row.jobId,
    status:  row.status  || 'running',
    step:    parseInt(row.step) || 1,
    message: row.message || ''
  }
}];
```

5.5 Node 4: Respond to Webhook — "Return Status JSON"

Respond Body:

```
{
  "jobId":   "{{ $json.jobId }}",
  "status":  "{{ $json.status }}",
  "step":    {{ $json.step }},
  "message": "{{ $json.message }}"
}
```

Expected Response Format

The HTML frontend reads these exact fields from every poll response:

status → "running" | "done" | "error"
step → 1, 2, 3, 4, or 5 (which step is currently active)
message → Optional text shown as subtitle on the loading screen

When status = "done", the frontend stops polling and shows the Thank You page.
When status = "error", the frontend stops polling and shows the Error page.

6. CORS Configuration

CORS (Cross-Origin Resource Sharing) must be enabled on both n8n webhooks. Without it, the browser will block all requests with a network error, even if n8n is running perfectly.

6.1 Enable CORS in n8n (Both Webhooks)

For each webhook node (Workflow 1 and Workflow 2), configure the following:

- In the Webhook node settings, find "Allow Multiple HTTP Methods" or "Options"
- Enable "Add Allowed Origins" and set value to: `*` (asterisk = allow all)
- Or if your HTML is hosted at a specific domain, use that domain for security

6.2 If CORS Cannot Be Set in n8n

Add a second Webhook node that responds to OPTIONS requests (preflight). Configure it like this:

```
// In n8n, add an HTTP Response node triggered by OPTIONS method
// Response headers to add:
Access-Control-Allow-Origin:  *
Access-Control-Allow-Methods: GET, POST, OPTIONS
Access-Control-Allow-Headers: Content-Type
```

7. Testing Checklist

Run through these tests in order before going live. Each test verifies a different layer of the system.

7.1 Pre-Launch Checklist

Check Item	How to Verify
WEBHOOK_URL is correct in HTML	Open HTML file, check line ~242, URL matches n8n Workflow 1 path
STATUS_POLL_URL is correct in HTML	Open HTML file, check line ~246, URL matches n8n Workflow 2 path
n8n Workflow 1 is Active (not paused)	In n8n, click Workflows → check green Active toggle on Workflow 1
n8n Workflow 2 is Active (not paused)	In n8n, click Workflows → check green Active toggle on Workflow 2
Google Sheets is connected in n8n	Run Workflow 1 manually and check if a row appears in the sheet
Workflow 1 returns jobId	Use Postman or browser console: POST the webhook and check response
Workflow 2 returns status JSON	Use browser: open STATUS_POLL_URL?jobId=test and check JSON
CORS is enabled on both webhooks	Browser DevTools → Network tab → check no CORS errors on request
Gmail node is connected and sending	Run full test with real email and verify delivery

7.2 Test Scenarios

Test Case 1 — Happy Path (Normal flow):

10. Fill in valid website URL, competitor URL, and your own email address
11. Click "Analyze Content Gaps"
12. Verify loading screen appears with Step 1 active
13. Open n8n → Executions tab → watch steps fire in real time
14. Verify UI step indicators advance as n8n progresses
15. After ~60–90 seconds, verify Thank You page appears
16. Check your inbox — report email should have arrived

Test Case 2 — No jobId Fallback:

17. Temporarily remove the jobId return from Workflow 1 "Respond" node
18. Submit the form
19. Verify the UI falls back to timed animation (does not crash)
20. A warning appears in browser DevTools console: "No jobId returned"

Test Case 3 — Error Handling:

21. In Workflow 1, add a node that throws an error mid-way
22. In the status update code, set status = "error"
23. Verify the UI shows the Error page with the correct message
24. Verify the Retry button works and restarts the process

8. Troubleshooting Guide

Symptom	Likely Cause	Fix
UI shows Thank You in 5 seconds	No jobId returned from n8n; fallback animation running	Check that Workflow 1 "Respond" node sends { jobId: ... } in response body
Steps 2–5 never activate on screen	Status Workflow 2 is not updating step values	Check Google Sheets is being written; check Workflow 2 is reading correct column
"Cannot reach server" error	Webhook URL is wrong or workflow is paused	Verify URL in HTML matches n8n path; check workflow is Active, not paused
CORS error in browser console	n8n does not have CORS headers enabled	Enable "Allow All Origins" in webhook settings in n8n
jobId received but polling never starts	STATUS_POLL_URL is wrong or unreachable	Check STATUS_POLL_URL in HTML; test it manually in browser
Thank You page never appears	Workflow never sends status="done"	Add final Code Node in Workflow 1 that sets status="done" after email node
Error page appears immediately	n8n returned HTTP 4xx or 5xx	Check n8n execution logs for errors; verify all credentials are valid
Email never arrives	Gmail node error or report generation failed	Check n8n execution logs; verify Gmail OAuth is still valid and not expired
Polling stops after 3 minutes	MAX_POLL_TIME_MS reached (timeout)	Process took too long; optimize n8n workflow OR increase MAX_POLL_TIME_MS in HTML

9. Quick Reference

9.1 Two URLs to Configure in HTML

```
WEBHOOK_URL     = your n8n Workflow 1 trigger webhook URL
STATUS_POLL_URL = your n8n Workflow 2 status check webhook URL
```

9.2 n8n Workflow 1 Response (must include jobId)

```
{ "jobId": "any_unique_string_here" }
```

9.3 n8n Workflow 2 Status Response (exact fields)

```
{
  "status":  "running"  // or "done" or "error"
  "step":    3          // number 1 through 5
  "message": "Optional detail text shown in UI"
}
```

9.4 Status Values Reference

status value	What the UI Does
"running"	Continues polling; updates step indicator and progress bar to current step
"done"	Stops polling; shows checkmark animation; navigates to Thank You page
"error"	Stops polling; marks current step as failed (red); navigates to Error page

9.5 Files Delivered

File	Purpose
seo-gap-analyzer-v2.html	The updated frontend with real-time polling. Deploy this to replace the old file.
SEO-Gap-Analyzer-SOP.docx	This document. Full reference for setup, configuration, and troubleshooting.

End of Document — SEO Gap Analyzer System SOP v2.0